
Building Back-End with Express.js

CONTENTS

Catalog

01

Introduction to Back-End
Mandate

02

Express Core: Routing &
Request Cycle

03

Security & Best Practices

04

Production & Performance
Optimization



01

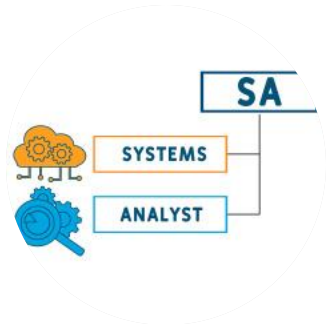
Introduction to Back-End Mandate

Goal: Scalable APIs for SaaS



Building scalable, high-performance APIs

Designing APIs that can handle growth and high traffic for SaaS applications



SaaS API Contract Definition

Using RESTful naming conventions to define clear and consistent API endpoints

Node.js Foundation Features



Asynchronous I/O

Utilizing Node.js's non-blocking I/O operations for efficient handling of concurrent requests



V8 Engine Power

Leveraging the V8 JavaScript engine for high-speed execution of JavaScript code

Express.js: Simplifying Node.js

Fast, minimalist framework

Express.js streamlines development by providing a simple and flexible set of features for Node.js applications

Simplifying Node.js Development

Express.js abstracts away the complexities of Node.js, allowing developers to focus on building features





02

Express Core: Routing & Request Cycle

Routing: API Contract Definition

01

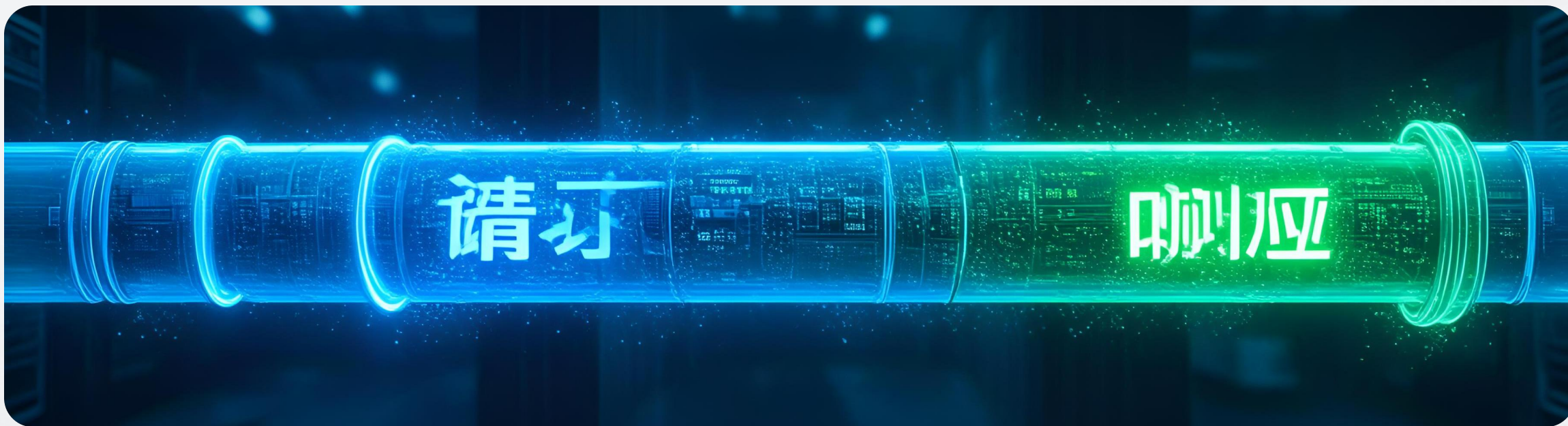
RESTful Naming Conventions

Using nouns and plurals to define API endpoints like
GET, POST, PUT, DELETE.

02

API Contract Definition

Routing in Express.js establishes the contract
between client and server.



Request-Response Cycle Process

HTTP Request Processing

Express processes incoming HTTP requests through the request-response cycle.

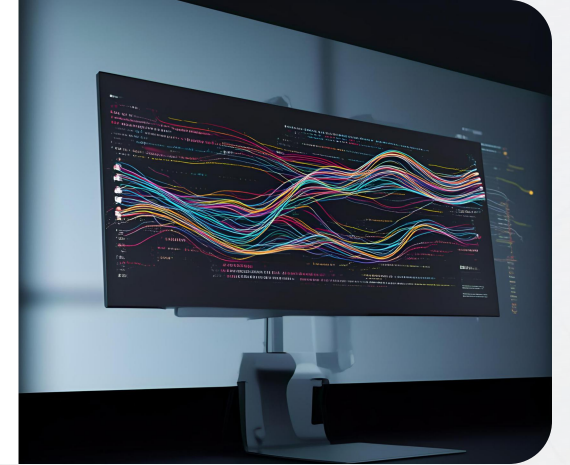
Intercepting Requests

Middleware intercepts and manipulates requests before reaching the route handler.

Middleware Power & Built-in Types

Intercepting and Manipulating Requests

Middleware functions like `express.json` and `express.static` handle requests.



Built-in Middleware Types

Express provides essential middleware types for common tasks such as parsing JSON.



03

Security & Best Practices

Authentication & Authorization

01

JWT Middleware

Using middleware for security checks, such as JSON Web Tokens (JWT).

02

Protecting API Endpoints

Ensuring that only authenticated and authorized users can access specific routes.

Essential Security Middleware

Setting Critical HTTP Headers

Applying Helmet middleware to prevent attacks like Cross-Site Scripting (XSS).



Rate Limiting Mechanism

Implementing rate limiting to protect against brute-force attacks and service abuse.



Code Reliability Measures



Asynchronous Programming Mandate

Avoiding synchronous functions to prevent blocking the main thread and maintain responsiveness.



Confidential Safety Complaint



Dedicated Error Handling

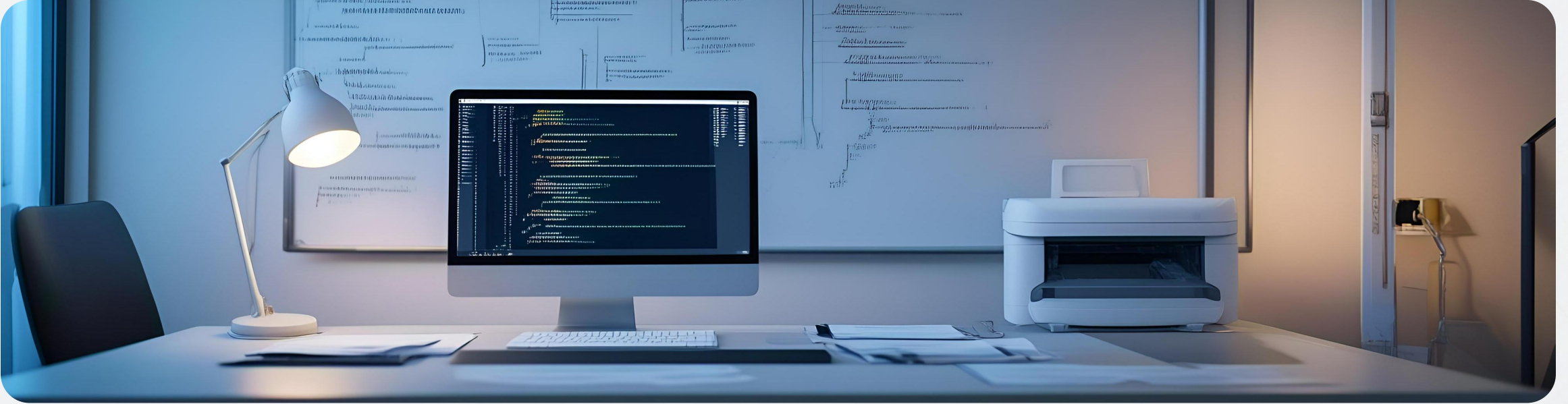
Utilizing error-handling middleware for graceful failure and improved user experience.



04

Production & Performance Optimization

Code Structure Organization



Separating application logic

Organize routes, controllers, and models to enhance maintainability and scalability.

Modular design

Implement a modular structure to improve code reusability and testability.

Performance Boosting Techniques



Setting NODE_ENV=production

Optimize performance by setting environment variables for production use.



Enabling Gzip Compression

Reduce response sizes and improve load times with Gzip compression.



Clustering/Process Managers

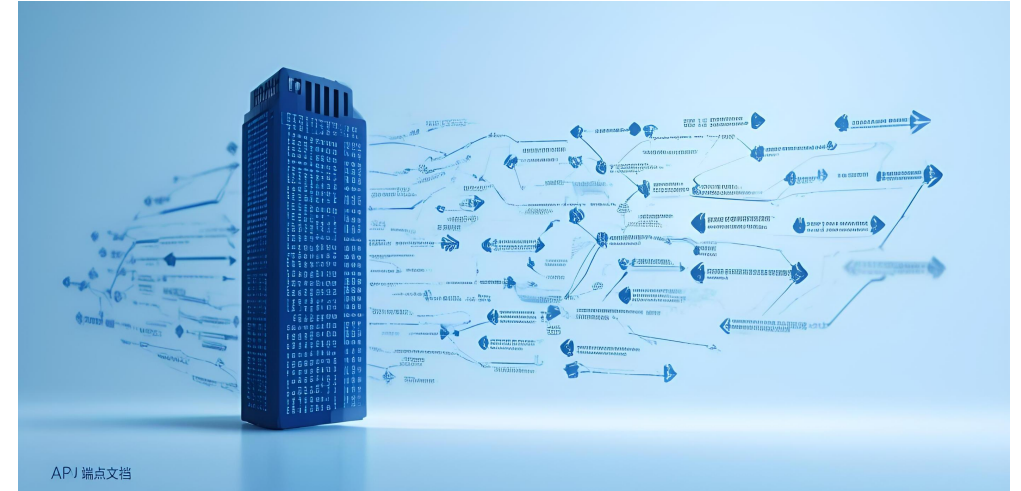
Utilize PM2 for load balancing and high-traffic handling.

API Documentation Standards



Standard response codes

Adopt standard HTTP status codes (200, 400, 401) for clear communication.



Documenting endpoints

Provide comprehensive documentation for each API endpoint to ensure clarity and usability.

Conclusion: Express for SaaS Back-End



Scalable, reliable back-end

Express.js offers a scalable and reliable solution for building SaaS back-ends.



Suits saascv applications

Express is particularly well-suited for the needs of SaaS applications.

THE END

Thank You